Welcome to your next lesson: Logical operators: and/or/not and short-circuiting.

I'm getting your personalized lesson ready, based on what you've learned already and what's coming up next, so you can keep building your skills.

Let’s dive in — you’re making real progress.

-

Imagine a swimming pool that has a deep end ?

To swim in the deep end, the rule is:

- You must be at least 12 years old
- and you must be at least 140 cm tall

So both things must be true at the same time.

In Python, we could check that like this:

Python

```
age = 13
height = 145

if age >= 12 and height >= 140:
    print("You can swim in the deep end")
```

Here `age >= 12` is one test, `height >= 140` is another test, and `and` connects them into one bigger condition.

Does this idea of using `and` for “both conditions must be true” make sense so far?

Yes, it makes sense

What do you think? ?

Given this code:

```
age = 10
height = 150

if age >= 12 and height >= 140:
    print("Deep end")
```

Will it print anything?

A.

Yes, it will print "Deep end"

B.

No, it will not print anything

Nice work, Ron ? You nailed that.

It does not print anything because with `and` both parts must be true:

Python

```
age >= 12      # 10 >= 12 → False
height >= 140  # 150 >= 140 → True
```

`False and True` is still `False`, so the `if` block is skipped and nothing prints.

Next, I want to show you how `or` is different (only one condition needs to be true).

Ready to see an `or` example next?

Yes, I'm ready

Awesome ? Let's bring in `or`.

Think about free shipping in an online store:

- You get free shipping if you are a member
- or your cart total is at least $50

So only one of these needs to be true.

In Python:

```python
is_member = False
cart_total = 60

if is_member == True or cart_total >= 50:
    print("Free shipping")
```

Here:

- `is_member == True` → False
- `cart_total >= 50` → True

`False or True` → overall condition is True, so `"Free shipping"` prints ✅

Does using `or` as "at least one condition is true" feel clear so far?

Yes, that's clear

Great ?

Now let's flip things and let you drive a bit.

Imagine this rule for a game ?:

- You can enter the tournament if your score is at least 80
- or your rank is 1

Here are some values:

```python
score = 75
rank = 1
```

Your task (in the code editor, not here in chat):

1. Write an `if` that uses `or` to check that rule.
2. Inside the `if`, `print("You can enter the tournament")`.
3. Run it, then Submit code.

Once you've submitted, tell me how it behaved: did it print the message or stay silent?

Python

```
score = 75
rank = 1
if score >= 80 or rank == 1:
    print("You can enter te tournament.")
```

Nice job, Ron ? Your code is correct, and that output shows your or condition is working:

```
score = 75      # score >= 80 → False
rank = 1        # rank == 1   → True

# False or True → overall True → print runs
```

So even though the score is too low, the rank being 1 is enough to let them enter.

Next I want to show you a visual way to think about how and and or decide True/False in different combinations.

Ready to look at that next?

Yes, I'm ready

Nice ?

Here's a simple picture in table form showing how and and or behave.

Think of two small tests:

- A (like age >= 12)
- B (like height >= 140)

We'll see what A and B and A or B become:

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Key idea:

- and → only True when both are True.
- or → True when at least one is True.

Does this table match how you're thinking about and vs or, or is any row confusing?

It makes sense

Great ?

Let's add one more tool: not.

not simply flips a True/False value:

- **not True** → False
- **not False** → True

Example:

Say a movie is allowed only if you are NOT under 13.

Python
```
age = 10

if not age < 13:
    print("You can watch the movie")
```

- age < 13 is **True** (10 < 13)
- not age < 13 becomes **False**

So the if is False and nothing prints.

Does this idea of not = “the opposite of this condition” feel clear?

Yes, it's clear

Nice ✅

Now let’s move to the second big goal of this lesson: short-circuiting.

Step by step:

- Python checks a condition from left to right.
- Sometimes it already knows the final answer from the left side.
- When that happens, it skips running the right side. That skipping is called short-circuiting.

Example with and:

Python
```
if False and (5 > 1):
    print("This will never run")
```

Python sees **False and** .. → with and, once the left side is **False**, the whole thing must be **False**, so it never even looks at 5 > 1.

Does that basic idea — “Python may stop early and skip the right side” — make sense so far?

Yes, it makes sense

Great, then let’s see short-circuiting in action using print ?

Look at this code:

Python
```
left = False
right = True

if left and print("Checking right side..."):
    print("Final block")
```

Step by step:

- Python evaluates left first → **False**

- For `and`, once the left side is **False**, the whole condition is already **False**
- So it never runs `print("Checking right side...")`
- Result: the program prints nothing at all

  The skipped `print` is how we prove short-circuiting happened.

  Before we try your own version: is it clear why `"Checking right side..."` does not appear for this code?

  > yes thats clear whats not is what the print() function would return had the left ide bee true, it rturns a 1 or 0 stating it was sucessfu i printing?

  Great question — you're thinking exactly the right way here ?

  In Python, `print()` doesn't return `1` or `0`. It actually returns a special value called `None`. So:

  ```
  Python
  result = print("Hello")
  ```

- This will show `Hello` on the screen
- But **result** will not be `1` or `0` — it will be `None`

  So in this kind of condition:

  ```
  Python
  if left and print("Checking right side..."):
      ...
  ```

- If `left` were **True**, then `print("Checking right side...")` would:
- print the text
- return `None` (which counts as False in an `if` test)

  So the overall condition would still be False, and `"Final block"` would not run — but that's outside our current focus. For now, we're just using `print()` as a visible sign: "did the right side run or get skipped?"

  Want to try a tiny experiment yourself: write code that uses `or` with a `print()` on the right side, and see when that `print` does or doesn't run?

  > i understand , wrte good thats what i was expecting some form of false so a print after the whole if this and that woul never happen as the print is also always false (none)

  Exactly, you've got the right picture ?

  In a line like:

  ```
  Python
  if left and print("Checking right side..."):
      print("AFTER whole if")
  ```

- `print("Checking right side...")` always returns `None`, which behaves like False in an `if`.
- So the full condition can never be True:

- If left is False → **False** and (anything) → False
- If left is True → **True** and **None** → still False

So **print**("AFTER whole if") will never run in that exact pattern.

We're only using **print**() inside the condition as a probe to see:

did Python even evaluate the right side, or did short-circuiting skip it?

Want to do a quick hands-on check with or now, to see when the right-side **print**() runs vs gets skipped?

Yes, let's try it

Nice ? Let's use or to see short-circuiting.

Here's the key idea with or:

- If the left side is already **True**, Python skips the right side (because **True or** anything is True).

I want you to test that yourself in the code editor:

1. Type this code:

Python
```
left = True
right = False

if left or print("Checking right side..."):
    print("Finished condition")
```

2. Run it.
3. Then change left to **False**, run again, and see what changes.
4. When you're happy with it, hit Submit code.

Don't tell me the output yet; just run both versions and submit the code you ended on.

After that, tell me: in which case did "Checking right side..." actually print?

Python
```
left = False
right = False

if left or print("Checking right side..."):
    print("Finished condition")
```

Perfect experiment, Ron ?

Your code:

Python
```
left = False
right = False

if left or print("Checking right side..."):
    print("Finished condition")
```

What happens step by step:

1. left is False
2. With or, Python can't decide yet (because False or ? could become True or stay False)
3. So it must evaluate the right side → runs print("Checking right side...")
4. print(...) returns None (False), so the condition is False or False → overall False
5. That means "Finished condition" does not print

So you correctly hit the case where the right-side print does run ✅

If you now changed:

```
Python
left = True
```

then:

- Python sees True or ... and short-circuits
- It skips print("Checking right side...")
- So "Checking right side..." would not appear at all.

I want to make sure the whole picture is solid:

- and → left False → skip right
- or → left True → skip right

Does that summary match how you're seeing it now, or is there any part of and/or/not or short-circuiting you'd like to zoom in on a bit more?

first run printed Finished condition with left = True

Good observation catching that first run ✅

Let's walk it slowly, because this is a great short-circuit example.

When you had:

```
Python
left = True

if left or print("Checking right side..."):
    print("Finished condition")
```

Step by step:

1. Python checks left → that's True.
2. For or, as soon as the left side is True, the whole condition is guaranteed True (True or anything → True).
3. Because of that, Python does not run print("Checking right side...") at all. It short-circuits.
4. Since the condition is True, it runs the body and prints:
   Finished condition

So:

- With left = True → "Finished condition" prints, "Checking right side..." does not.

- With left = **False** → "Checking right side..." prints, "Finished condition" does not.
  So the right-side print only runs when Python still needs that information to decide the result.
  Let me check your mental model:
  Does this rule feel clear now?
- and: left is False → skip right
- or: left is True → skip right
  Or would you like to see one more tiny example with and to lock it in?

  It's clear now

  Love it ? You've built a solid mental model.
  Quick recap of what you now understand:
- and
- Both sides must be True.
- If the left is False, Python skips the right side.
- or
- At least one side must be True.
- If the left is True, Python skips the right side.
- not
- Just flips True ↔ False.
- Using print() on the right side is a probe to see if that part is even evaluated (short-circuiting), not to get a True/False result from print() itself.
  Two quick checks so I know you're fully ready to move on:

1. If you see:
```
if is_raining and print("Taking umbrella check"):
    print("Go outside")
```
   When is_raining is **False**, will "Taking umbrella check" print?
2. If you see:
```
if is_admin or print("Logging non-admin access"):
    print("Access granted")
```
   When is_admin is **True**, will "Logging non-admin access" print?
   You can just answer:

1. yes/no
2. yes/no

Then, if those look good, I'll suggest we wrap this lesson and head to the next one ?

1. No short circuit xits as is_raining is False
2. No as it will also short circuit because its got a true ondtion o left so doesnt run right however #2 will print access grnted whereas #1 will not

Exactly right on both counts, Ron ?

Your reasoning:

1. `if is_raining and print("Taking umbrella check"):`

- `is_raining` is False → with `and`, Python stops immediately → right side skipped.
- Condition is False → inner `print("Go outside")` doesn't run.
- Result: nothing prints at all ✅

2. `if is_admin or print("Logging non-admin access"):`

- `is_admin` is True → with `or`, Python stops immediately → right side skipped.
- Condition is True → `print("Access granted")` runs.
- Result: you only see `Access granted` ✅

You've nailed:

- Combining conditions with `and`, `or`, `not`
- And using `print()` to prove short-circuiting on the right side

? You've fully mastered this lesson.

When you're ready, you can click Next Lesson in the Learn tab to keep going, or if you want, we can do a quick extra challenge using these ideas in a small real-world scenario.

How would you rate this lesson?